

Neural Networks with Möbius Basel-Check Memory Transformer Attention, Inverse-Score Weighting, Harmonic Steps, and Long-Chain Thought

Unified Framework for Spectral Compression and Invariant Validation

Abstract

We present a unified framework integrating neural networks with a memory architecture based on the Möbius function and the Basel checksum. The memory stores coefficients at odd square-free indices, compresses signals via the chip pipeline (convolution + supertrace + Möbius compression), and uses the Basel sum

$$\sum_{n=1}^{\infty} \frac{\mu(n)}{n^2} = \frac{6}{\pi^2}$$

as an invariant to detect drift, prevent infinite loops, and bound network atrophy. The natural step $a = 1/(\pi - e) \approx 2.362$ emerges from the identity $a^{-1} + e = \pi$ and serves as the derivative step and error tolerance. We extend this framework with a transformer architecture whose attention weights are monomial hashes over square-free indices, computed via an inverse-score (merge-sort inversion count) that yields a scalar importance for each token. Harmonic numbers provide the natural discrete time step for neuron updates. Memory and cache are organised in chunks derived from word importance, with asymmetric reconstruction that uses the Möbius ASCII logic gate for further compression. A pre-allocated buffer enables long-chain thought through the learned pendulum dynamics. All operations run in $O(K \log K)$ time.

1 Introduction

The combination of number theory and neural computation offers a novel approach to memory management and signal processing. The Möbius function $\mu(n)$, defined as

$$\mu(n) = \begin{cases} 1 & \text{if } n \text{ is square-free with an even number of prime factors,} \\ -1 & \text{if } n \text{ is square-free with an odd number of prime factors,} \\ 0 & \text{if } n \text{ has a squared prime factor,} \end{cases}$$

provides a natural filter for spectral coefficients. The Basel sum

$$\sum_{n=1}^{\infty} \frac{\mu(n)}{n^2} = \frac{6}{\pi^2}$$

gives the density of square-free numbers and acts as a checksum for memory integrity.

In this work we build a complete neural architecture around these ideas. The transformer layer uses monomial hash attention (Section 3), where each token is embedded as

a sparse polynomial over square-free indices. Attention weights are scalars derived from the inverse score (Section 4), a normalised inversion count that measures the ordering relationship between two embeddings. Harmonic numbers serve as the natural discrete time step for neuron updates (Section 2.2). Memory and cache are stored in chunks derived from word importance, with asymmetric reconstruction and Möbius ASCII compression (Section 5). Finally, a pre-allocated buffer enables long-chain thought via the learned pendulum dynamics (Section 6). Throughout, the constant $a = 1/(\pi - e)$ unifies derivative steps, entropy scaling, and error tolerances.

2 Mathematical Framework

2.1 Möbius Sieve

The linear sieve computes $\mu(1), \dots, \mu(K)$ in $O(K)$ time:

$$\begin{aligned}\mu(1) &= 1, \\ \mu(n) &= - \sum_{d|n, d < n} \mu(d) \quad \text{or via the linear sieve.}\end{aligned}$$

2.2 Harmonic Numbers

The harmonic numbers

$$H_n = \sum_{k=1}^n \frac{1}{k}$$

serve as the discrete time coordinate for the spectral sum $\zeta(t)$:

$$\zeta(t) = \sum_{i=-K}^K \mu(|i|) \exp\left(\frac{i t i}{\alpha}\right), \quad \alpha = \frac{1}{\pi - e}.$$

In the neural network, each neuron integrates its input over the harmonic time steps H_n . The update rule for a neuron with state h is

$$h_{n+1} = h_n + \frac{1}{H_n} (\text{input}_n - h_n),$$

which mimics the natural decay and integration of synaptic signals.

2.3 Supertrace and Entropy

For a signal C of length N , the supertrace is

$$S = \sum_{k=1}^N (-1)^{k+1} |C_k|,$$

the entropy is

$$H = -\alpha \frac{|S|}{N} \log\left(\frac{|S|}{N}\right),$$

and the invariant mass is

$$m = |S| e^{-H}.$$

The natural step $a = 1/(\pi - e)$ sets the error tolerance:

$$\Delta S \leq a \cdot N.$$

2.4 Basel Checksum

The Basel checksum is defined as

$$B(K) = \sum_{n=1}^K \frac{\mu(n)}{n^2}.$$

For large K , $B(K) \rightarrow 6/\pi^2$. The density of non-zero Möbius values is

$$\lim_{K \rightarrow \infty} \frac{|\{n \leq K : \mu(n) \neq 0\}|}{K} = \frac{6}{\pi^2}.$$

This serves as an invariant check for memory compression.

3 Transformer with Möbius Monomial Attention

We adopt the attention mechanism from `attention-monomial-hash-elliptic.py`. Each token is embedded as a sparse polynomial:

$$\phi(t) = \sum_{n \in \mathcal{S}} c_n(t) x^n,$$

where $\mathcal{S}$ is the set of square-free indices ($\mu(n) \neq 0$), and $c_n(t)$ is a coefficient derived from the token’s hash. The query, key, and value are all such sparse embeddings.

Given a query embedding Q and a set of key embeddings $\{K_i\}$, the attention weight between Q and K_i is computed as

$$\alpha_i = \text{softmax}\left(-\frac{1}{\tau} \text{inv}(Q, K_i)\right),$$

where $\text{inv}(Q, K_i)$ is the inverse score (see Section 4), and τ is a temperature. The output is a weighted sum of the value embeddings:

$$\text{Attention}(Q, \{K_i\}, \{V_i\}) = \sum_i \alpha_i V_i.$$

Because the embeddings are sparse over square-free indices, the inverse score can be computed in $O(K \log K)$ using merge-sort inversion counting on the common support.

4 Inverse-Score Weighting

The inverse score from `ML.py` is defined for two sequences a and b of length K :

$$\text{inv}(a, b) = \frac{\text{number of inversions in } b \text{ when sorted by } a}{K(K-1)/2}.$$

This yields a scalar in $[0, 1]$, where 0 means perfect forward agreement and 1 means perfect inverse agreement. In the neural network, this scalar is used as a weight for the connection between two neurons or between a token and an attention head. Specifically, the weight w_{ij} between neuron i and neuron j is

$$w_{ij} = 1 - \text{inv}(h_i, h_j),$$

so that neurons with similar ordering have strong positive weights, while those with inverse ordering have strong negative weights. This provides a biologically plausible, number-theoretic alternative to gradient-based weight updates.

5 Memory, Cache, and Asymmetric Reconstruction

The memory architecture is built on the `MöbiusCollatzMemory` and the chip compression pipeline. The memory stores key-value pairs (n, v) where n is odd and square-free ($\mu(n) \neq 0$). The Collatz step transforms indices:

$$n \mapsto \frac{3n+1}{2^k}, \quad k = v_2(3n+1),$$

keeping only odd square-free results.

5.1 Word Importance and Chunking

Each word in a sentence is assigned an importance scalar s_w using the inverse score between the word’s embedding and the sentence embedding:

$$s_w = \text{inv}(E_w, E_{\text{sentence}}).$$

Words are then grouped into chunks of size C according to their importance. High-importance words are stored in dedicated chunks, while low-importance words are compressed more aggressively.

5.2 Asymmetric Reconstruction

When reconstructing from memory, the decoder uses the importance scalars to decide how many coefficients to retrieve for each word. For a word with importance s_w , the number of coefficients kept is

$$M_w = \max(1, \lfloor |S| \cdot (1 - s_w) \rfloor),$$

where S is the supertrace of the chunk. This yields an asymmetric reconstruction: important words are reconstructed with high fidelity, while less important words are approximated.

5.3 Möbius ASCII Logic Gate

For further compression, we apply the Möbius ASCII logic gate from `hex-ok.py`. Each chunk is passed through the gate:

$$\text{chunk} \xrightarrow{\text{ASCII}} \text{polynomial} \xrightarrow{\text{elliptic perm}} \text{Möbius filter} \xrightarrow{\text{logic gate}} \text{chip compression}.$$

The output is a compact hexadecimal fingerprint that can be stored in the cache. The Basel checksum is recomputed after unpacking to detect corruption.

5.4 Cache Hierarchy

The cache is organised as a two-tier hierarchy:

- **SSD layer:** stores the real part of the $|C_i|$ array via memory-mapped files.

- **RAM layer:** caches the imaginary part, computed on demand as

$$\text{im}[i] = -\frac{d(\text{re})}{dn} \cdot \frac{1}{a},$$

with $a = 1/(\pi - e)/0.3628 \approx 6.511$.

The analytic envelope $\text{env}[i] = \sqrt{\text{re}[i]^2 + \text{im}[i]^2}$ is used for supertrace and entropy computations. This cache design allows efficient access to derived quantities without storing them permanently.

6 Long-Chain Thought via Buffer

To enable long-chain thought, we employ the pre-allocated buffer from `pendulum-long-chain-buffer.py`. The buffer holds a state vector $h \in \mathbb{R}^4$ representing the pendulum state:

$$h = (x_{\text{inv}}, y_{\text{inv}}, x_i, y_i).$$

The dynamics are governed by the learned pendulum cell:

$$\frac{dh}{dt} = f(h, \zeta),$$

where ζ is the driving input from the spectral sum. The buffer is updated in place, avoiding dynamic memory allocation:

$$h_{t+1} = h_t + \Delta t f(h_t, \zeta_t).$$

This allows the network to maintain a coherent train of thought over thousands of steps. The buffer size is $2K + 1$, matching the length of the $|C_i|$ array, and the supertrace of the buffer state provides a measure of the thought’s intensity.

7 Neural Network Integration

7.1 Inference Buffer

The buffer holds a signal array and provides:

- `apply_gate(gate_name)`: log, exp, sin, cos, derivative
- `compress()`: chip compression
- `collatz()`: Collatz index transform
- `inverse_score_with(target)`: inversion count ($O(K \log K)$)

7.2 Atrophy Prevention

The inverse score between current activations and a reference is monitored. When it exceeds the threshold 0.3, a correction is applied:

$$\mathcal{L}_{\text{corr}} = \text{MSE}(h_{\text{cur}}, h_{\text{ref}}) \cdot \min(1, H_{\text{ent}}/0.5).$$

This mimics biological homeostasis.

7.3 Derivative Gate

The derivative operator

$$D_a f(t) = \frac{f(t+a) - f(t)}{a}, \quad a = \frac{1}{\pi - e},$$

is implemented as a logic gate, matching the finite-difference method from the Ruby code.

8 Compression and Serialization

8.1 Ternary Packing

Each Möbius value is packed into 2 bits:

$\mu(n)$	Code
0	00
1	01
-1	10

The compressed size is $\lceil K/4 \rceil$ bytes.

8.2 Basel Validation

After unpacking, the Basel sum is recomputed. If it deviates from $6/\pi^2$ beyond a tolerance, the memory is flagged as corrupted.

9 Convergence and Bounded Loops

The `MobiusConvergence` class applies gates, compression, and Collatz steps iteratively until the signal norm change falls below `tol` or `max_iter` is reached. The loop is bounded by `max_iter` $\times O(K \log K)$.

10 Applications

- **Text processing:** ASCII signals are compressed via the Möbius gate.
- **Collatz reinforcement learning:** SuperTrace reward + inverse-score match.
- **Cross-compilation:** Code lines are hashed into spectral coefficients.
- **Harmonic oscillator simulation:** Step gates enumerate the oscillator in $O(K \log K)$.
- **Long-chain reasoning:** The buffer enables coherent thought over extended sequences.

11 Conclusion

The Möbius Basel-check memory provides a robust, number-theoretic foundation for neural network memory management. The natural constant $a = 1/(\pi - e)$ unifies the derivative step, entropy scaling, and error tolerance. The Basel checksum ensures integrity, prevents infinite loops, and bounds drift. The transformer attention with monomial hashes and inverse-score weights offers a sparse, interpretable alternative to dense attention. Harmonic numbers provide a natural time step for neuron updates. Memory and cache are organised in importance-derived chunks with asymmetric reconstruction and Möbius ASCII compression. The pre-allocated buffer enables long-chain thought. All operations are $O(K \log K)$, making the framework scalable and practical.

References

1. Möbius function and the Basel problem: `basel-check-error.py`
2. Spectral sum and harmonic points: `harmonic-alpha-0k.py`
3. Chip compression pipeline: `chip-g.py`, `mobius-gates.py`
4. Neural atrophy prevention: `atrophy.py`
5. Derivative operator: `derida.rb`
6. Collatz memory: `random-access-colla-mobius.py`
7. Monomial hash attention: `attention-monomial-hash-elliptic.py`
8. Inverse score: `ML.py`
9. Möbius ASCII logic gate: `hex-ok.py`
10. Long-chain buffer: `pendulum-long-chain-buffer.py`
11. SSD multi-layer cache: `ssd-ram-ui.py`